LGWallRecon

Light-guide inner-wall reconstruction
from an entrance-face scan

User manual

Hengne Li (李衡讷)

South China Normal University

Hengne.Li@cern.ch Hengne.Li@m.scnu.edu.cn

11 November 2026

In one paragraph. LGWallRecon simulates a hollow conical light guide with a mirror wall by ray tracing, and searches for a wall shape for which the simulated response map matches a measured scan of the entrance face. For the scan shipped with the package the simulation and the measurement differ by 0.9 % in the inner aperture, against about 0.8 % point noise. The wall that achieves this is *not unique*: the scan fixes only a small part of the wall shape. Read the wall figure as the minimum unevenness needed to explain the scan, not as a survey of the real surface.

Contents

1 What the toolkit does **3**

2 Physics and model **5**

2.1 Geometry and coordinates . . . 5
2.2 Source, wall and detector . . . 5
2.3 The ideal cone . . . 5
2.4 The wall as a B-spline surface . . . 6
2.5 Ray–wall intersection . . . 6
2.6 Quasi-random rays and common random numbers . . . 6
2.7 How a patch of wall shows up in the scan . . . 7
2.8 The inversion . . . 7

3 Installation **8**

3.1 Requirements . . . 8
3.2 Python environment . . . 8
3.3 GPU (optional but recommended) . . . 8
3.4 Geant4 (optional) . . . 9

4 Quick start **9**

5 The pipeline, step by step **10**

5.1 `edge.py`: the entrance aperture . . . 10
5.2 `edge_spot.py`: the light spot . . . 10
5.3 `stageA.py`: the axisymmetric profile . . . 10
5.4 `refine.py`: re-gridding along z . . . 11
5.5 `kernel_modes.py`: the linear response . . . 11
5.6 `stageB.py`: the two-dimensional inversion . . . 11
5.7 `final.py`: the high-statistics comparison . . . 12
5.8 `pipeline.sh`: the whole chain . . . 12
5.9 Exports, figures and the report . . . 12

6 Validation tools **13**

6.1 Geant4 . . . 13
6.2 GPU against CPU . . . 14
6.3 Reciprocity . . . 14
6.4 Jacobian test . . . 14
6.5 Closure test . . . 14
6.6 Resolution study . . . 15
6.7 Further studies . . . 16

7 Outputs and file formats **16**
7.1 Input scan 16
7.2 Intermediate files in `out/` 17
7.3 Results 17

8 Adapting the toolkit to another light guide or scan **17**

9 Limits and interpretation **18**

10 Troubleshooting **19**

11 Resource policy: `tools/run_limited.sh` **20**

12 Author, contact and citation **20**

1 What the toolkit does

The light guide is a hollow cone, 17.5 mm in diameter at the entrance and 8 mm at the exit, 100 mm long, lined with reflective film. In the measurement a small diffuse light source is moved over the entrance face on a 100×100 grid with 0.2 mm step, and a PMT at the exit records the signal at each position. The result is a two-dimensional response map. It is not flat: it shows an off-centre dark spot, a bright ring and several bright patches, a few percent in size.

LGWallRecon answers two questions.

1. **Can a ray-tracing simulation reproduce this map?** Yes. An ideal cone responds identically at every source position, so all the structure must come from the wall departing from the ideal shape. With a suitably deformed wall the simulated map agrees with the measured one to 0.9 % (rms, inner aperture $r < 8$ mm, 5029 scan points); the ideal cone misses by 5.4 %. The point noise of the measurement is about 0.8 %.
2. **Which wall does it?** The toolkit returns one: a deviation field $\delta(\varphi, z)$ from the CAD cone with a local unevenness of 13 µm rms, largest bump −49 µm, largest dent +72 µm, on top of an azimuthal mean deviation between −0.25 and −0.95 mm (the bore is narrower than CAD).

The honest headline. That wall is not the only one. In a closure test a known wall with seven bumps and dents was turned into a synthetic scan with the measured noise and sent through the same pipeline. The synthetic map was reproduced down to the noise (residual 0.80 %), yet the recovered wall correlates with the true wall at only 0.09: true local unevenness 48 µm, recovered 11 µm. A linear analysis explains why. The scan is sensitive to roughly 100 combinations of the wall shape. Inside that visible subspace the recovered and the true wall correlate at 0.72–0.97; but the test wall had only about 5 % of its deformation power there. The rest is imprinted on the scan too weakly to be recovered.

How to read the wall figure. It is the smoothest wall, with the least unevenness, among those that reproduce the scan. It tells how much unevenness is needed at the least, and where the scan-visible part of it sits. It is not a survey of the real inner surface. Section 9 lists what extra measurements would be needed for that.

What the package contains:

- a specular ray tracer for a cone with a free-form wall, on the CPU (`numba`) and on the GPU (OpenCL), tracing the same rays;
- the inversion pipeline: aperture measurement, axisymmetric profile, linear-response kernels, damped least-squares inversion with exact residuals;
- validation tools: Geant4 cross-check, reciprocity test, GPU-against-CPU test, Jacobian test, closure test, resolution study;
- figure, table and report generators in Chinese and English, including a single-file web page with a live 3-D demonstration of rays in the guide.

Figure 1 shows the wall returned for the shipped scan and Figure 2 the comparison of the simulated with the measured map.

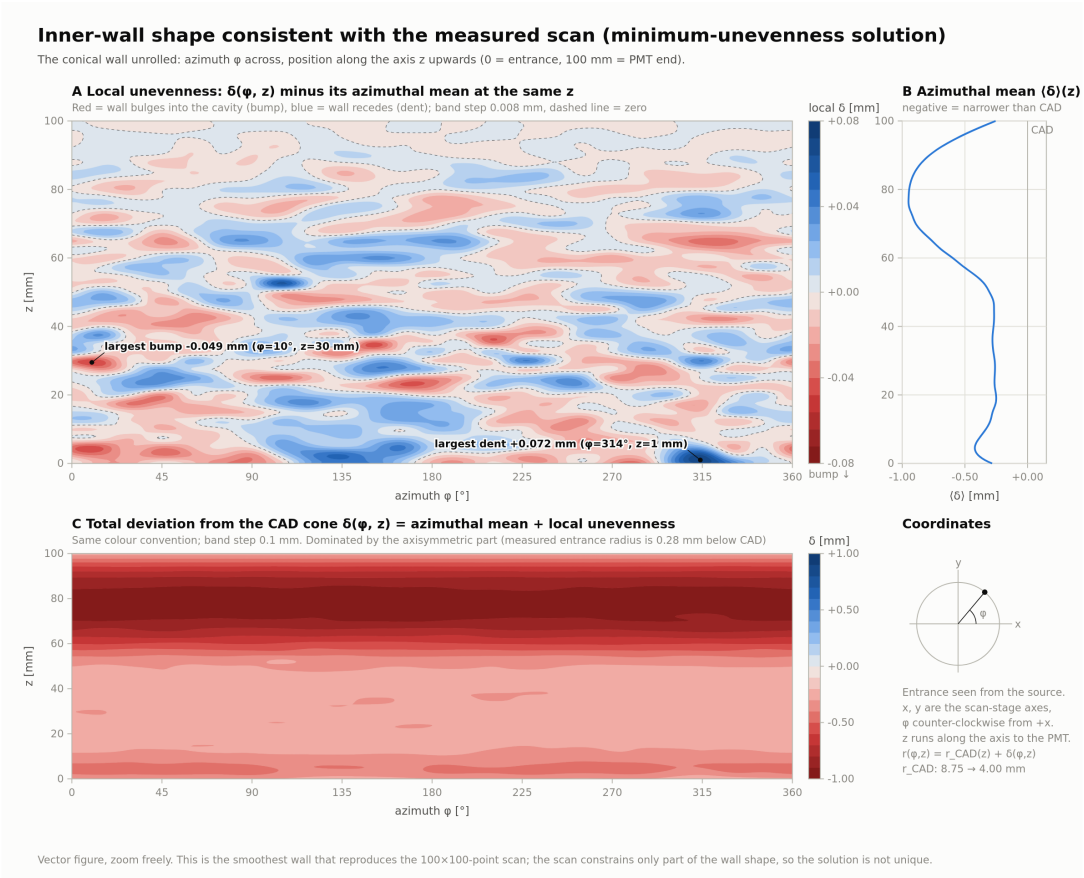


Figure 1: The wall figure (`inner_surface.svg`). A: local unevenness, the deviation minus its azimuthal mean. B: the azimuthal mean. C: the total deviation from the CAD cone. Red is a bump into the cavity, blue a dent.

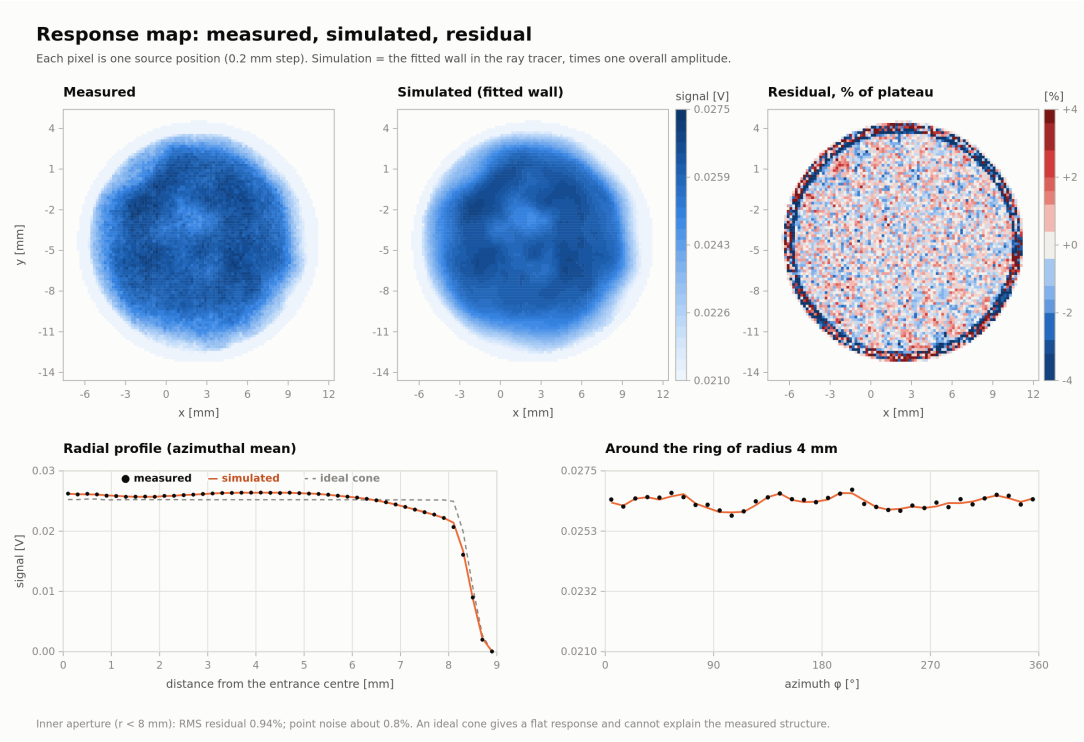


Figure 2: Measured map, simulated map and residual (`response_maps.svg`), with the azimuthally averaged radial profile and the signal around the ring of radius 4 mm.

2 Physics and model

2.1 Geometry and coordinates

All lengths are in millimetres. The z axis is the optical axis, $z = 0$ at the entrance plane and $z = L = 100$ at the exit plane where the PMT sits. The nominal (CAD) wall radius is

$$r_{\mathrm{CAD}}(z) = R_1 + (R_2 - R_1)\,\frac{z}{L}, \qquad R_1 = 8.75, \quad R_2 = 4.00,$$

a cone of half angle 2.72°. The real wall is written as

$$r(\varphi, z) = r_{\mathrm{CAD}}(z) + \delta(\varphi, z).$$

$\delta < 0$ means the wall bulges into the cavity (a bump); $\delta > 0$ means it recedes (a dent).

The transverse axes x, y are the scan-stage axes, with the origin moved to the measured centre of the entrance aperture. The azimuth φ is counted counter-clockwise from $+x$, looking at the entrance from the light-source side. Relating φ to the hardware requires knowing how the stage axes are oriented with respect to the light guide, and which of the four cells of the block was scanned; the toolkit cannot know either.

The CAD file `data/4cells_17.5mm_8mm_circle_to_circle_100mm-Cut.step` describes a block with four identical cells. Each cell is the cone above followed by a cylindrical pocket of 16.36 mm diameter and 10 mm depth that holds the PMT. Only the cone enters the simulation.

2.2 Source, wall and detector

Source A small spot in the entrance plane that emits like a Lambertian surface into the forward hemisphere. The spot is a uniform disc; its radius (0.34 mm for the shipped scan) is measured from the sharp edge of the scan. Rays that start outside the aperture are blocked.

Wall A perfect mirror: specular reflection with reflectivity 1. A reflectivity below 1 can be set (parameter `refl`) and was tried as a free parameter; it came out at 0.94 without removing the need for geometric deformation and without a clearly better fit, so results are quoted for a perfect mirror.

Detector Every ray that crosses the exit plane $z = L$ inside the wall is counted, whatever its angle and position. A ray that returns through the entrance plane is lost.

The simulated signal at source position (x, y) is $A\,T(x, y)$, where T is the fraction of all emitted light that reaches the exit and A is one overall amplitude (volts per unit transmission) fitted to the data. The absolute scale of the measurement is therefore not used.

2.3 The ideal cone

For a diffuse point source the ideal cone has a remarkably simple response: the transmission is the same at every source position,

$$T_{\mathrm{ideal}} = \left(\frac{R_2}{R_1}\right)^2 = 0.209.$$

This is conservation of étendue: a lossless concentrator cannot pass more than the area ratio of a Lambertian input, and a long shallow cone reaches that limit everywhere. The tracer reproduces 0.2090 to three digits and Geant4 gives 0.209 ± 0.002. The consequence for the inversion is important: any non-uniformity of the measured map must come from the wall being non-ideal (or from effects outside the model, see Section 9).

2.4 The wall as a B-spline surface

$\delta(\varphi, z)$ is a tensor-product uniform cubic B-spline,

$$\delta(\varphi, z) = \sum_{i=0}^{N_\varphi - 1} \sum_{j=0}^{N_z + 2} c_{ij}\, B_i^{(\varphi)}(\varphi)\, B_j^{(z)}(z).$$

In φ the spline is periodic with $N_\varphi = 24$ knots at $\varphi_i = 2\pi i / N_\varphi$ (15° apart). In z there are N_z cells on $[0, L]$ and $N_z + 3$ coefficients; coefficient j is centred at $z_j = (j - 1)\, L / N_z$, so the first and the last one sit one cell outside the guide. The final fit uses $N_z = 40$ (2.5 mm cells, 43 coefficients per azimuth), which gives $24 \times 43 = 1032$ wall parameters. The spline is C^2, so the surface normal is continuous, which specular reflection needs.

2.5 Ray–wall intersection

A ray $\mathbf{p} + t\,\mathbf{d}$ hits the wall where $f(t) = \sqrt{x^2 + y^2} - r_{\mathrm{CAD}}(z) - \delta(\varphi, z)$ changes sign from negative to positive. The tracer finds the first such t in three steps.

1. *Bounding cones.* δ lies between two numbers δ_{lo} and δ_{hi} (sampled on a fine grid, with a small margin). The wall therefore lies between the cones $r_{\mathrm{CAD}} + \delta_{\mathrm{lo}}$ and $r_{\mathrm{CAD}} + \delta_{\mathrm{hi}}$, whose intersections with the ray are roots of a quadratic. They bracket the hit.
2. *Marching.* The bracket is walked in steps of at most 3 mm until f changes sign.
3. *Illinois regula falsi* on that last step, to 10^{-10} mm in double precision (2×10^{-6} mm in single precision).

The outward normal is the gradient of $F = r - r_{\mathrm{CAD}}(z) - \delta(\varphi, z)$,

$$\mathbf{n} \;\propto\; \hat{\mathbf{r}} \;-\; \frac{1}{r}\frac{\partial \delta}{\partial \varphi}\,\hat{\boldsymbol{\varphi}} \;-\; \Big(\frac{R_2 - R_1}{L} + \frac{\partial \delta}{\partial z}\Big)\hat{\mathbf{z}},$$

with the derivatives taken analytically from the spline, and the ray is reflected as $\mathbf{d}' = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\,\mathbf{n}$. A ray ends when it crosses $z = L$ (detected) or $z = 0$ (lost).

2.6 Quasi-random rays and common random numbers

For a Lambertian source the direction cosines (d_x, d_y) are uniform in the unit disc. Only directions with $\sin\theta \le s_{\max}$ are sampled, because steeper rays never reach the exit (for the fitted wall nothing is lost with $s_{\max} = 0.5$; the pipeline uses 0.55), and the result is scaled by $s_{\max}^2$.

Rays are quasi-random rather than random. The four numbers that define a ray (two for the direction, two for the position inside the spot) come from the R_4 additive recurrence $u_j(k) = \mathrm{frac}(s_j + k\, g^{-j})$ with $g = 1.2207\ldots$, evaluated in 32-bit integer arithmetic so that the CPU and the GPU produce the same rays. Each source position has its own shifts s_j (a Cranley–Patterson rotation derived from a hash of the point index and a seed). Two properties follow.

- The noise of T falls faster than $1/\sqrt{N}$: with 20 000 rays per point it is about 0.2–0.4 % of T, where independent random rays would give about 1.5 %.
- For a fixed seed the map is a deterministic function of the wall. Differences between two walls are then computed with *common random numbers*: most rays have the same fate in both, and the noise of the difference is far below the noise of either map. All finite differences and all fit iterations rely on this.

2.7 How a patch of wall shows up in the scan

A small deformation at height z and azimuth φ does not change the response near the point on the entrance face "below" it. It changes the fate of the marginal rays, those that just make it to the exit or just fail to, and for a given wall patch these rays come from a thin arc of source positions. The arc is typically 0.5 mm wide, and its position and radius change quickly with z: patches near the entrance affect sources at the adjacent rim, patches around mid-length draw arcs and rings a few millimetres from the centre, patches near the exit give diffuse rings. The picture resembles tomography, with each scan pixel summing wall deformations along a curve.

Two practical consequences: the response is very sensitive (a deformation of order 0.03 mm changes the map by a few percent), and it needs a fine axial resolution of the wall model (2.5 mm cells) to place the arcs correctly.

2.8 The inversion

The fit minimises

$$\chi^2(c, A) = \sum_n \frac{\big(d_n - A\,T_n(c)\big)^2}{\sigma^2} + \lambda^2\,\|Pc\|^2 + (0.05\,\lambda)^2\,\|c - c_{\text{base}}\|^2 + \sum_i \frac{\big(\delta(\varphi_i, 0) - \delta_{\text{ap}}(\varphi_i)\big)^2}{(0.005\ \text{mm})^2}.$$

d_n are the pedestal-subtracted scan values inside the fit radius (8 mm by default), σ combines the point noise (2.0×10^{-4} V) with the Monte Carlo noise of the model, P takes first differences of the coefficients along φ and z (a smoothness penalty), c_{base} is the axisymmetric base wall, and the last term ties the wall radius at $z = 0$ to the aperture radius measured from the edge of the scan.

$T_n(c)$ is not differentiable in closed form, and a brute-force finite-difference Jacobian is too noisy for single spline coefficients. The package uses two devices.

Fourier-mode kernels. Around an axisymmetric base wall the response to an azimuthal mode $\delta c_{ij} = a\cos(m\varphi_i)$ at one z knot is a sum of a few harmonics $\sum_q A_{jmq}(\rho)\cos(q\psi)$ of the source azimuth ψ, with $q \in \{m,\, N_\varphi - m,\, N_\varphi + m\}$. One run perturbs the whole circumference, so the signal is strong while the wall slopes stay small, and projecting on $\cos(q\psi)$ averages the noise over all source angles. Central differences remove the second-order term. The Jacobian for any single coefficient follows by Fourier synthesis and rotation. Against exact finite changes of the map these kernels reach a correlation of 0.91–0.98 and a slope of 0.93–1.0.

Damped iterations with exact residuals. Each Levenberg–Marquardt step solves the linearised problem with a damping term $\mu\|\Delta c\|^2$, then evaluates the true χ^2 with a full ray trace. The step is kept if χ^2 falls (and μ is halved); otherwise μ is multiplied by four and the step is recomputed, up to eight times. A Broyden rank-one update corrects the Jacobian with what each trial step actually did to the map. Because the residual is always exact, kernel errors slow the convergence but do not bias the answer.

The axisymmetric part of the wall is large (tenths of a millimetre) and non-linear, so the chain first fits an axisymmetric profile alone (Stage A), then runs two rounds of kernels and inversion, the second with kernels recomputed around the updated azimuthal mean.

3 Installation

3.1 Requirements

Component	Needed for
Python $\geq$ 3.10 with `numpy`, `scipy`, `matplotlib`, `numba`, `contourpy`, `pillow`	everything
`pyopencl` and an OpenCL GPU	fast tracing (optional; without it use `LG_GPU=0`)
Geant4 11.x, CMake, a C++17 compiler	the independent cross-check in `g4/` (optional)
Node.js	the animated ray figure `ray_animation.svg` (optional)
TeX Live with `xelatex` and `ctex`	rebuilding this manual (optional)
Firefox	rendering the manual figures from SVG (optional)
Linux with `systemd` user sessions	the resource wrapper `tools/run_limited.sh`

3.2 Python environment

```
git clone <repository url> LGWallRecon
cd LGWallRecon
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
```

The scripts expect the interpreter at `.venv/bin/python` relative to the package root. The shell drivers accept another interpreter through the variable `PY`.

3.3 GPU (optional but recommended)

The tracer runs on any OpenCL device that `pyopencl` can see. Check with

```
clinfo -l
.venv/bin/python -c "import pyopencl as cl; print([d.name for p in cl.get_platforms()
    for d in p.get_devices()])"
```

The development machine has an AMD Radeon 8060S (Ryzen AI Max+ 395) driven through ROCm 7.2; the device appears as `gfx1151` on the platform “AMD Accelerated Parallel Processing”. On AMD hardware `nvidia-smi` is absent and says nothing about GPU availability; use `clinfo` or `rocminfo`.

Measured speeds for the fitted wall:

Configuration	Rays per second
GPU, single precision (default)	about 5×10^7
GPU, double precision	about 12 times slower than single
CPU, 23 cores, double precision (`numba`)	about 2.5×10^6

The GPU version is therefore about twenty times faster than the CPU version on this machine.

The kernel is single precision; it was checked ray by ray against the CPU in double precision (Section 6). Without a GPU everything still runs with `LG_GPU=0`; expect the full chain to take several hours instead of half an hour.

Integrated GPUs share the system memory, and a cgroup memory limit does not cap GPU allocations. The tracer keeps its buffers at a few megabytes and submits work in slices of about half a second, so it neither exhausts memory nor trips the driver watchdog.

3.4 Geant4 (optional)

```
source <geant4 prefix>/bin/geant4.sh
cd g4 && mkdir -p build && cd build
cmake .. -DCMAKE_PREFIX_PATH=<geant4 prefix>
make -j4
```

This builds `g4/build/lgsim`. No Geant4 data sets beyond the standard installation are needed; the program uses a minimal physics list with optical photons, transportation and the optical boundary process only.

4 Quick start

```
cd LGWallRecon
./run_all.sh
```

`run_all.sh` measures the aperture, runs the reconstruction chain on `data/scan_summary.csv`, exports the wall, draws the figures in both languages and assembles the report pages. Every heavy step goes through `tools/run_limited.sh` (Section 11).

Expected run time with the GPU: about 8–12 minutes for each of the two kernel computations, one to two minutes for each inversion, half a minute for the final map; 25–30 minutes for the whole chain.

Where the results appear:

Path	Content
`results/en/inner_surface.svg`, `results/zh/...`	the wall figure (vector, zoomable)
`results/<lang>/response_maps.svg`	measured map, simulated map, residual, profiles
`results/<lang>/closure.svg`	closure test (after `closure.sh`)
`results/<lang>/ray_animation.svg`	animated side view of traced rays
`results/wall_delta.csv`	$\delta(\varphi, z)$ on a $1^\circ \times 0.5\,\mathrm{mm}$ grid
`results/wall_coeffs.json`	the B-spline coefficients
`results/numbers.json`	every number quoted in the report and in this manual
`docs/LGWallRecon_report_en.html`, `..._zh.html`	single-file report pages with the 3-D demonstration
`out/`	intermediate `.npz` and `.log` files (not versioned)

To run the stages by hand, change into `src/` first; the scripts use paths relative to it.

```
cd src
R=../tools/run_limited.sh; PY=../.venv/bin/python
$R $PY edge.py && $R $PY edge_spot.py      # aperture and light spot -> ../out/edge.npz
$R ./pipeline.sh data                      # the reconstruction chain
```

```
$R ./make_report.sh ../out/final_data.npz  # exports, figures, report pages
```

5 The pipeline, step by step

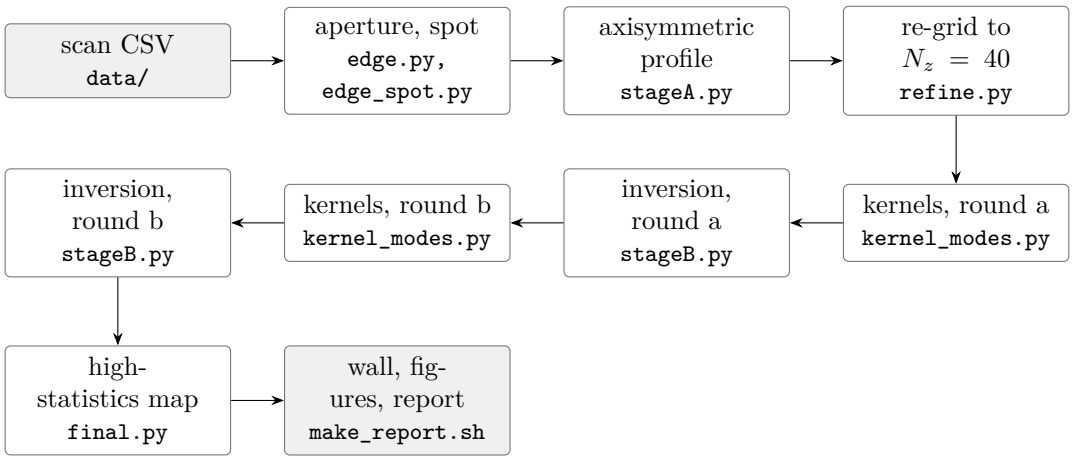


Figure 3: The reconstruction chain. `pipeline.sh` runs the boxes from `stageA.py` to `final.py`.

All commands below are run from inside `src/`, through `../tools/run_limited.sh`. Environment variables used by the scripts are collected in Table 1.

5.1 `edge.py`: the entrance aperture

```
python edge.py
```

The edge of the scan is sharp, so the aperture can be measured directly, without ray tracing. The script takes the pedestal as the median signal far outside the aperture ($r > 10\,\mathrm{mm}$), then fits the radial edge profile in 48 azimuthal sectors with a plateau level, a slope and an error-function edge. The centre is moved until the $m = 1$ Fourier component of the edge radius vanishes. Output: `../out/edge.npz` with the centre, the pedestal and the Fourier coefficients ($m = 0 \ldots 8$) of the edge radius $r_{\mathrm{ap}}(\varphi)$, and a diagnostic plot `../out/edge.png`.

For the shipped scan the mean radius is 8.47 mm, that is 0.28 mm below the CAD value of 8.75 mm, with $m = 2$ and $m = 4$ components of about 0.03 mm.

The first guess of the centre and the radial window of the edge fit are constants near the top of the script; change them for a different aperture (Section 8).

5.2 `edge_spot.py`: the light spot

```
python edge_spot.py
```

A global fit of all edge points compares a Gaussian spot with a uniform disc. The disc describes the edge better (the signal has no tail outside the aperture). The script adds the disc radius (0.34 mm) and a small correction of the mean aperture radius to `../out/edge.npz`. All later stages read the spot from that file.

5.3 `stageA.py`: the axisymmetric profile

```
LG_TAG=data python stageA.py 10 fix 3.0
```

Arguments: number of z cells N_z (10), `fix` or `refl` (fit the mirror reflectivity as well), weight of the curvature penalty (3.0). The script fits an axisymmetric wall offset $\delta_0(z)$ and the amplitude to the azimuthally averaged radial profile of the scan ($\rho = 0.1 \ldots 7.9\,\mathrm{mm}$), by Levenberg–Marquardt with finite differences and common random numbers (120 000 rays per radius). The radius at $z = 0$ is tied to the measured aperture. Output: `../out/A_<tag>.npz` and a plot.

How to judge it: the printed "data" χ^2 should fall by more than an order of magnitude from the start value, and the residual plot should stay within about $\pm 1\,\%$ of the plateau. The profile itself is *not* well determined: different runs return visibly different $\delta_0(z)$ with the same fit quality (Section 9).

5.4 `refine.py`: re-gridding along z

```
python refine.py ../out/A_data.npz 40 ../out/start_data.npz
```

Re-expresses a wall on a different number of z cells. The input is either a Stage-A result (the profile is copied to all 24 azimuthal knots) or a fitted wall. For nested grids ($10 \to 40$) the conversion is exact.

5.5 `kernel_modes.py`: the linear response

```
python kernel_modes.py ../out/start_data.npz 24 dataa 12000 0.06
```

Arguments: a wall file (its azimuthal mean becomes the base wall), N_φ, a tag, rays per source point (12 000), mode amplitude in mm (0.06; modes above $m = 3$ use $0.06 \cdot 3/m$). Sources sit on a polar grid of 57 radii (0.075 to 8.475 mm) and 49 angles on the half circle. For each of the 43 z knots and each mode $m = 0 \ldots 12$ the script traces the wall with $+a$ and $-a$ and stores the harmonic amplitudes $A_{jmq}(\rho)$. Output: `../out/kmodes_<tag>.npz`. Run time: about 10 minutes on the GPU; one progress line per z knot.

The module also provides `jacobian(kn, rho, psi)`, which assembles the Jacobian for arbitrary source positions from the stored amplitudes.

5.6 `stageB.py`: the two-dimensional inversion

```
LG_MU=3000 python -u stageB.py dataa dataa 1.0 40 ../out/start_data.npz
```

Arguments: kernel tag, output tag, regularisation weight λ (1.0), maximum number of iterations (40), start wall. Output: `../out/fitB_<outtag>.npz`, rewritten after every accepted iteration, so an interrupted run can be continued by passing that file as the start wall.

One line is printed per iteration:

```
it 7: chi2/N 1.518  rms resid 1.016 %  A 0.14979  |delta-base| rms 0.063 max 0.199
    rough 11.291
```

- `chi2/N`: data χ^2 per scan point, with σ = point noise $\oplus$ model noise. A value near 1 means the map is fitted to the noise. The shipped scan ends at 1.31: a few tenths of a percent of structure on a 1 mm scale is not explained by the model. The synthetic scan of the closure test reaches 1.0, so the remainder is a property of the data, not of the optimiser.

- `rms resid`: rms of data minus model, in percent of the mean signal.
- `|delta-base|`: rms and maximum of the coefficients relative to the base wall, in mm.
- `rough`: the value of $\|Pc\|^2$.

The run ends after the given number of iterations or with "`no improving step; stop`". That message means that eight successive damped steps, down to sub-micrometre size, failed to lower the true χ^2: the fit sits at a local minimum for the present kernels. It is the normal way for the second round to finish. If it appears in the very first iteration with a large χ^2, the damping start value is too small for the state of the wall; raise `LG_MU` (the chain uses 3000 for round a and 20 000 for round b).

With `LG_VERB=1` every trial step is printed with its size, the χ^2 predicted by the linear model and the χ^2 actually achieved. A large gap between the two is the signature of kernels that no longer describe the wall; recompute them around the current azimuthal mean.

5.7 `final.py`: the high-statistics comparison

```
python final.py ../out/fitB_datab.npz data 40000
```

Traces the whole scan grid with the fitted wall (two seeds, to measure the model noise) and with the ideal cone of the measured aperture radius, fits the amplitude, prints the summary numbers and writes `../out/final_<tag>.npz` and a diagnostic PNG. All figures and the report are built from this file.

5.8 `pipeline.sh`: the whole chain

```
../tools/run_limited.sh ./pipeline.sh data
```

Runs Stage A, the re-gridding, two rounds of kernels and inversion, and `final.py`, with the settings given above. The tag names all intermediate files (`A_data`, `start_data`, `kmodes_dataa`, `fitB_dataa`, `kmodes_datab`, `fitB_datab`, `final_data`). `edge.py` and `edge_spot.py` must have been run before. With `LG_SYNTH=<npz>` the chain runs on a synthetic scan instead of the measured one.

5.9 Exports, figures and the report

Command	Result
`export_wall.py <final npz> <dir>`	`wall_delta.csv`, `wall_coeffs.json`
`collect_numbers.py <final npz> <results dir>`	`numbers.json` (also reads the Geant4, reciprocity, closure and resolution results if present)
`make_figures.py <final npz> <dir> <lang>`	`inner_surface.svg`, `response_maps.svg`
`make_closure_fig.py <synth npz> <final npz> <dir> <lang>`	`closure.svg`, `closure_numbers.json`
`make_ray_svg.py <final npz> <dir> <lang>`	`ray_animation.svg` (needs Node.js)
`build_page.py <final npz> <results dir> <out html> <lang> [github] [gitlab]`	one single-file report page
`make_report.sh <final npz>`	all of the above for `zh` and `en`

<lang> is zh or en. All strings shown in figures and pages live in web/i18n/zh.json and web/i18n/en.json; the code contains no display text.

Table 1: Environment variables.

Variable	Used by	Meaning (default)
LG_GPU	lgtrace.py	1: trace on the GPU; 0: CPU. The wrapper sets 1; without the wrapper the default is 0
LG_CPUS	run_limited.sh	number of CPU cores (chosen from the idle cores, 4–24)
LG_MEM	run_limited.sh	hard memory limit of the job (6G)
LG_SCAN	load_scan.py	path of the scan CSV (data/scan_summary.csv)
LG_SYNTH	stageA.py, stageB.py, final.py	.npz with a synthetic scan that replaces the measured one
LG_TAG	stageA.py	name of the Stage-A output, A_<tag>.npz
LG_RCUT	stageB.py	fit radius in mm (8.0)
LG_NRAY	stageB.py	rays per scan point in each iteration (20 000)
LG_MU	stageB.py	start value of the damping (3000)
LG_BROYDEN	stageB.py	1: secant correction of the Jacobian (1)
LG_VERB	stageB.py	1: print every trial step (0)
PY	shell drivers	Python interpreter (../.venv/bin/python)

6 Validation tools

The checks answer three different questions: is the tracer right, can the inversion recover a known answer, and how much does the result depend on the assumptions. Table 2 collects the results for the shipped scan.

Table 2: Validation results.

Check	Result	Verdict
Geant4, ideal cone	$T = 0.209 \pm 0.002$, flat	pass; theory 0.2090
Geant4, fitted wall	ratio 1.0019 ± 0.0011	pass; 120 points × 4000 photons, normalised scatter 1.08
GPU against CPU	identical ray by ray	pass; at most 1 ray per point differs in double precision
Reciprocity	ratio 1.003 ± 0.008	pass
Jacobian test	correlation 0.91–0.98	pass; slope 0.93–1.0
Closure test, the map	residual 0.80 %	pass; fitted to the noise
Closure test, the wall	correlation 0.09	**fails**; 48 µm true, 11 µm recovered
Free reflectivity	$R = 0.94$	no clear improvement; geometry still needed

6.1 Geant4

g4/lgsim traces optical photons in a hollow cavity whose wall is an optical surface of type dielectric_metal, polished, with the given reflectivity. The cavity is either the ideal G4Cons or a G4TessellatedSolid built from a wall mesh.

```
lgsim <mesh.txt | ideal> <points.txt> <photons per point> <out.txt> [spot] [smax] [refl
    ] [seed]
```

`spot` > 0 is a Gaussian sigma, < 0 a uniform disc of radius $-$`spot` (mm); `smax` restricts the launch cone as in the tracer. The output has one line per source point: x, y, detected, emitted, T.

Ideal cone.

```
cd src && python test_ideal_cone.py                # tracer: T(rho), flat at 0.209
cd ../g4 && build/lgsim ideal run/radial.txt 20000 run/ideal.out 0 0.7 1.0 1
```

Fitted wall.

```
cd g4
./run_final.sh ../out/fitB_datab.npz       # mesh export, 12 parallel jobs, comparison
```

The mesh has 1440×251 vertices, about 720 000 triangles; facets are 0.25° wide and 0.4 mm long, fine enough that faceting does not show at the present precision. Tracking in the tessellated solid costs about 10 ms per photon (the ideal `G4Cons` needs 16 µs), which is why Geant4 is used for checking and not for fitting. `run_final.sh` starts one process per point list (12 lists of 10 points here), each of about 0.7 GB; 4000 photons per point take roughly ten minutes. `g4_compare.py` traces the same points with the tracer at high statistics and prints the ratio of the mean transmissions, the normalised scatter (1 for pure statistics) and the correlation.

Result: ratio 1.0019 ± 0.0011, normalised scatter 1.08, correlation 0.97.

6.2 GPU against CPU

```
python test_gpu_vs_cpu.py
```

Traces the same points on the CPU (double precision), on the GPU in double precision and on the GPU in single precision. In double precision the GPU agrees with the CPU to at most one ray per point. Single precision is unbiased (mean difference 2×10^{-7} in T) with a few rays per point changing fate, as expected for marginal rays.

6.3 Reciprocity

```
LG_GPU=0 python test_reciprocity.py ../out/fitB_datab.npz
```

For a lossless specular guide lit with the same Lambertian radiance from both ends, the flux from entrance to exit equals the flux from exit to entrance. A wrong intersection, or a normal that is inconsistent with the surface, breaks this identity. The script traces both directions on the CPU for a smooth wall, two random rough walls and the fitted wall. For the fitted wall the ratio is 1.003 ± 0.008; the forward flux also equals the exit area, as étendue requires.

6.4 Jacobian test

```
python test_jacobian.py dataa ../out/fitB_dataa.npz
```

Compares the kernel prediction $J\,\Delta c$ with the exact change of the map (200 000 rays per point, common random numbers) for random smooth perturbations of 0.01–0.1 mm rms, around the base wall and around a fitted wall.

6.5 Closure test

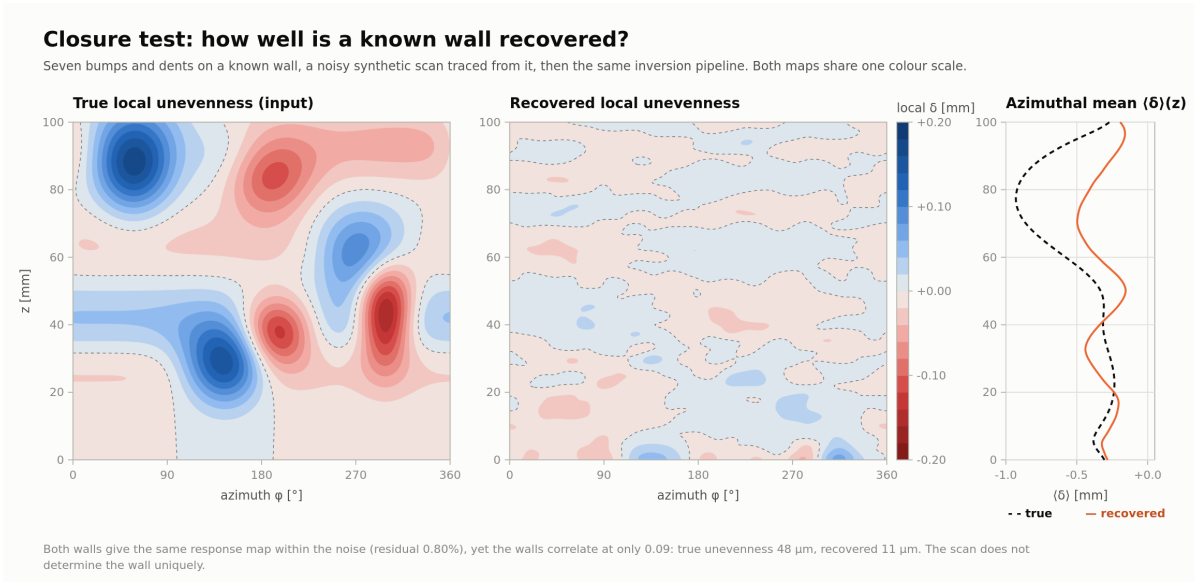


Figure 4: Closure test. Left: the true local unevenness put in. Middle: the unevenness recovered from the synthetic scan, on the same colour scale. Right: azimuthal mean profiles. Both walls give the same response map within the noise.

```
../tools/run_limited.sh ./closure.sh 1
python make_closure_fig.py ../out/synth_cl1.npz ../out/final_cl1.npz ../results/en en
```

`make_synth.py` builds a known wall: the Stage-A profile plus seven Gaussian bumps and dents (0.15–0.3 mm high, 2–4 mm wide along the arc, 6–12 mm long), on a finer spline grid than the fit uses, so the truth is not exactly representable. It traces a synthetic scan and adds the measured point noise. `closure.sh` then runs `pipeline.sh` on it.

The outcome (Figure 4) has two parts and both matter. The synthetic map is reproduced to the noise: residual 0.80 %, $\chi^2/N \approx 1$. The pipeline converges properly. The wall is not recovered: the correlation between true and recovered local unevenness is 0.09, the recovered amplitude is 11 µm against 48 µm true, and the azimuthal mean profile is off by 210 µm rms.

6.6 Resolution study

```
python study_resolution.py cl1b ../out/synth_cl1.npz ../out/fitB_cl1b.npz
```

A singular-value decomposition of the Jacobian orders the wall patterns by how strongly they change the scan. A pattern with singular value s_k is visible at one standard deviation if its coefficient amplitude exceeds σ_T/s_k, with σ_T the noise of one scan point in transmission units. Table 3 gives, for the leading k patterns, that smallest visible amplitude, the share of the true test wall's deformation power that lies in them, and the correlation between the true and the recovered wall inside them.

Table 3: What the scan can see (closure test). The spectrum falls by three orders of magnitude over the 1032 patterns.

Leading patterns k	Smallest visible amplitude [µm]	Share of true wall [%]	Correlation
10	1.1	0.2	+0.82
30	1.8	2.4	+0.97
60	3.3	2.7	+0.95
100	6.5	5.4	+0.72
150	11.3	9.3	+0.57
200	18.1	11.1	+0.49
300	39.2	15.7	+0.32
500	93.2	19.8	+0.25

The leading 30–150 patterns are seen down to 2–11 µm and are recovered with a correlation of 0.72–0.97, but they hold only about 5 % of the test wall's deformation. Beyond a few hundred patterns the scan is blind at the 0.1 mm level.

6.7 Further studies

Script	Purpose
`study_deformations.py`	response maps for a set of simple test deformations (barrel, hourglass, single bumps, entrance flare, exit pinch)
`study_reflectivity.py`	radial profile of the ideal cone for mirror reflectivities below 1, against the data
`study_launch_cone.py`	checks that the launch-angle cut `smax` loses no detected light
`look_data.py`	quick plots of the raw scan

7 Outputs and file formats

7.1 Input scan

`data/scan_summary.csv` has one line per scan point with the columns `idx_x`, `idx_y`, `direction`, `cmd_x_mm`, `cmd_y_mm`, `pos_x_mm`, `pos_y_mm`, `timestamp`, `n_events`, `n_saved`, `probe_V`, `passed`, `acq_time_s`. The grid indices `idx_x`, `idx_y` place each point; the code uses the positions `pos_x_mm`, `pos_y_mm`, the signal `probe_V` (volts) and the flag `passed`. The grid must be regular and complete.

7.2 Intermediate files in `out/`

File	Arrays
`edge.npz`	`cx`, `cy` centre (stage coordinates); `ped`, `ped_rms` pedestal; `fourier[9,2]` cosine and sine coefficients of $r_{\mathrm{ap}}(\varphi)$; `phi`, `r_edge`, `sigma`, `level` per-sector edge fits; `spot_radius`, `dr_disc` from `edge_spot.py`
`A_<tag>.npz`	`p` profile coefficients (and the reflectivity if fitted); `NZ`; `rho`, `dat`, `err`, `T`, `A` the fitted radial profile; `zz`, `d0` the profile $\delta_0(z)$
`start_<tag>.npz`	`c[24, NZ+3]`, `NZ`, `NPHI`, `refl`
`kmodes_<tag>.npz`	`A[nk, 13, 3, nrho]` harmonic amplitudes; `QS[13,3]` the harmonics q of each mode (-1 = unused); `rho`; `p` base profile; `T0` base map; `NPHI`, `NZ`, `refl`
`fitB_<tag>.npz`	`c[24, 43]` fitted coefficients; `base`; `A` amplitude; `hist` (χ^2/N, residual, amplitude per iteration); `T`, `xs`, `ys`, `dat` model, positions and data of the fitted points; `lam`, `NPHI`, `NZ`, `refl`
`final_<tag>.npz`	`x`, `y` scan axes; `S[nx,ny]` pedestal-subtracted data; `M` model; `T` transmission; `Mideal` ideal cone; `A`; `R` distance from the centre; `cx`, `cy`; `phi[361]`, `z[201]`, `DEL[361,201]` the wall on a grid; `c`; `refl`; `mc` model noise
`synth_<tag>.npz`	`S` synthetic scan; `c_true[48,23]`; `bumps`; `A`

7.3 Results

`wall_delta.csv`: three comment lines, then the columns `phi_deg` (0–359, step 1), `z_mm` (0–100, step 0.5), `r_cad_mm`, `delta_mm`, `delta_azimuthal_mean_mm`, `delta_local_mm`.

`wall_coeffs.json`: `NPHI`, `NZ` and the coefficient matrix `c[NPHI][NZ+3]` in mm. It can be fed back to the tracer:

```
import json, numpy as np, lgtrace as lg
c = np.array(json.load(open('../results/wall_coeffs.json'))['c'])
T = lg.transmission([0.0, 2.0], [0.0, -1.0], coeffs=c, nray=20000, smax=0.55, sig
    =-0.34)
```

`sig` < 0 is a disc of radius $-$`sig`; `sig` > 0 a Gaussian sigma; positions are relative to the aperture centre.

`numbers.json`: the fit quality (`rms`, `rms_id`, `chi2`, `corr`, `rms_ring`), the wall statistics (`loc_rms`, `loc_min`, `loc_max` in µm; `mean_min`, `mean_max` in mm), the aperture (`r_ap`, `spot_r`), and the blocks `g4`, `recip`, `closure`, `resolution` when those checks have been run.

Geant4 mesh (`mesh.txt`): first line "`NPHI NZ`"; then `NZ` lines, each with z and the `NPHI` wall radii at $\varphi_i = 2\pi i/$`NPHI`, in mm.

8 Adapting the toolkit to another light guide or scan

Geometry. The cone is defined by three constants, which appear in four places and must agree:

- `src/lgtrace.py`: `R1, R2, L` (and `SLOPE` derived from them);
- `src/lgcl.py`: the macros `R1`, `LEN`, `SLOPE` in the OpenCL source (`SLOPE` is written out as a number);
- `web/lgtrace.js`: `R1, R2, L` for the browser demonstration;

- `g4/lgsim.cc`: `R1, R2, LEN`, and the sizes of the world box and of the two photon dumps if the guide is much larger.

`export_wall.py` and `make_ray_svg.py` also write out $r_{\mathrm{CAD}} = 8.75 - 0.0475\,z$. A shape that is not a straight cone needs a different reference surface in the function `_f` and in the bounding-cone bracket; the B-spline part carries over unchanged.

Scan. Any regular rectangular grid works. Either provide a CSV with the same columns and point `LG_SCAN` to it, or edit `load()` in `src/load_scan.py`; it must return the axes `x`, `y` and the signal `V[ix, iy]`. In `src/edge.py` set the first guess of the centre and the radial windows used for the pedestal ($r > 10$) and for the edge fit ($7.6 < r < 9.6$).

Fit radius. `LG_RCUT` (8.0 mm) must stay inside the aperture by more than the spot radius, so that the spot is never clipped at a fitted point. The polar grid in `kernel_modes.py` (57 radii of 0.15 mm) must cover it.

Launch cone. `smax` must exceed the largest $\sin\theta$ of any detected ray, roughly R_2/R_1 plus a margin. Check with `study_launch_cone.py`. Too small a value silently loses light; too large a value only costs time.

Resolution of the wall. N_φ is an argument of `kernel_modes.py` and `refine.py` writes 24 azimuthal knots for a Stage-A input; N_z is the second argument of `refine.py`. Finer grids resolve thinner arcs but add poorly constrained parameters; cost grows with $N_z \times (N_\varphi/2 + 1)$ kernel runs.

Regularisation. λ is the third argument of `stageB.py`. With the damped iterations the result depends weakly on it: the fit stays close to the start wall anyway. To see the effect of the prior, change the start wall rather than λ.

Noise and statistics. The point noise of the scan (`SIGD` in `stageB.py`, also used in `final.py` and `collect_numbers.py`) is 2.0×10^{-4} V for the shipped data; estimate it for new data from the high-pass residual of the plateau. Rays per point: 12 000 for the kernels, 20 000 per iteration, 40 000 for the final map. Halving them roughly halves the run time and raises the model noise by 30–40 %.

9 Limits and interpretation

The solution is not unique. Different walls give the same response map. The scan constrains about 100 combinations of the wall shape, down to 1–10 µm; the trace left by broad, gentle bumps 0.2–0.3 mm high can equally be produced by a different wall with only about ten micrometres of unevenness. The damped iterations stay near the start wall, so the result is the wall with the least unevenness among those that fit. The local unevenness of 13 µm rms is a lower bound on what the real wall must have, not an estimate of what it has.

The azimuthal mean profile is poorly fixed. It is strongly degenerate with the overall signal scale and with the exit radius. Two runs on the same data returned profiles that differ by several tenths of a millimetre near the exit while fitting equally well. Reliable are only the entrance radius (8.47 mm, measured directly from the scan edge) and the statement that the bore is narrower than CAD.

Everything is attributed to geometry. By assumption the mirror reflectivity is 1, the PMT response is uniform in position and angle, and the source is an ideal Lambertian disc. Real reflectivity variations, the seam of the reflective film, dust, or photocathode non-uniformity would all be read as wall unevenness.

Unexplained structure. The fit to the shipped scan ends at $\chi^2/N = 1.31$, not 1: a few tenths of a percent of correlated structure on a 1 mm scale remains. The closure test shows that the pipeline reaches the noise when the truth lies inside the model, so this remainder is real structure that the model (smooth wall, perfect mirror, uniform detector) cannot produce.

The entrance rim. In the outermost 0.4 mm ring of the aperture the residual is about 5 % per point, and 2–4 % after azimuthal averaging. The measured signal drops earlier than the model inside the aperture but has no tail outside, as if a band about 0.1 mm wide at the rim were lossy (a chamfer, or film that stops short of the edge). This is below the resolution of the model and the ring is excluded from the fit.

Resolution. The deviation field has 2.5 mm cells along the axis and 15° in azimuth (2.2 mm of arc at the entrance, 1.0 mm at the exit). Finer structure is smoothed.

Orientation of the azimuth. φ follows the scan axes. Whether the figure is a mirror image of the hardware depends on the stage orientation.

What would pin down the real wall. Independent information, each piece constraining a further set of shape combinations:

- scans with a collimated beam at several incidence angles (each angle selects different ray families, where the diffuse source averages over all of them);
- angle-resolved or position-resolved readout at the exit instead of one integrating PMT;
- a scan from the exit side;
- direct profilometry of the wall, or a coordinate measurement of the bore at a few heights, to fix the azimuthal mean.

The resolution study (`study_resolution.py`) can be repeated for a simulated additional measurement to see how many more patterns it would make visible before taking the data.

10 Troubleshooting

`IndexError` or "no devices" when the GPU tracer starts

`pyopencl` found no GPU device. Check `clinfo -l`; make sure the user is in the `render` and `video` groups; on AMD check that the ROCm OpenCL ICD is installed. To run without a GPU set `LG_GPU=0`.

The desktop stutters or the GPU driver resets during a run

The tracer submits slices of about 0.6 s (`slice_s` in `lgcl.transmission`). Lower it if the GPU is shared with interactive work. Run only one GPU job at a time.

Results change after editing `lgtrace.py`, or `numba` complains about a cached function

Delete `src/__pycache__`. The compiled functions are cached there and rebuilt on the next run (a few seconds).

A job dies with exit code 137, or "Killed"

It exceeded the memory limit of the wrapper. The pipeline needs less than 1 GB; a Geant4 job with the 720 000-facet mesh about 0.7 GB. Raise the limit with `LG_MEM=12G` only if the job is known to need it.

"no improving step; stop" in the first iteration

See Section 5: raise `LG_MU`, or recompute the kernels around the current wall. At the end of a converged run the message is normal.

`stageB.py` converges, then the final χ^2 is higher

`final.py` uses a different seed and more rays; differences of a few hundredths in χ^2/N are Monte Carlo noise. Larger differences mean the fit followed the noise of its own ray set; raise `LG_NRAY`.

The manual figures come out empty, or Firefox refuses to start

Another Firefox is using the same profile. `make_manual_figs.sh` keeps a private profile in `docs/manual/.ffprofile`; delete that directory and run again.

Geant4: "Overlapping daughter with mother volume"

The world box in `lgsim.cc` must contain the cavity and both photon dumps; enlarge it when the guide is longer than 100 mm.

Geant4: "Particle e- without a Process Manager"

This appears when `G4OpticalPhysics` is registered alone. Keep the minimal physics list of `lgsim.cc`: the optical photon only, with transportation and the optical boundary process (`G4OpBoundaryProcess`).

Geant4 is very slow

Tracking in the tessellated solid costs about 10 ms per photon whatever the mesh size between 180 000 and 720 000 facets. Use few points with a few thousand photons, in parallel processes.

11 Resource policy: `tools/run_limited.sh`

The development machine is shared, so every long or parallel job is started through a wrapper that makes it impossible for a simulation to hurt the machine.

```
tools/run_limited.sh <command> [arguments...]
```

- The command runs in a transient `systemd` user scope with `MemoryMax` (default 6 GB) and no swap: a job that exceeds the limit is killed at once and nothing else is affected.
- The number of cores is chosen when the job starts: the idle cores minus four spare, between 4 and 24. It sets the CPU quota of the scope and the thread counts of `numba`, OpenMP and BLAS.
- The job runs at `nice 19` with idle I/O priority, so other work always wins.
- `LG_GPU=1` is exported, so tracing runs on the GPU unless `LG_GPU=0` is given.

Overrides: `LG_MEM=12G`, `LG_CPUS=8`, `LG_GPU=0` in front of the command. On a machine without `systemd` user sessions run the commands directly and set `LG_GPU=1` and `NUMBA_NUM_THREADS` by hand.

The GPU is outside the cgroup accounting. The tracer therefore limits itself: host buffers of a few megabytes and sub-second slices. Run one GPU job at a time.

12 Author, contact and citation

LGWallRecon was written by Hengne Li (李衡讷), South China Normal University.

Contact: `Hengne.Li@cern.ch`, `Hengne.Li@m.scnu.edu.cn`.

Repositories:

- GitHub: `https://github.com/hengne/LGWallRecon-Claude-Fable52`
- GitLab (SCNU group): `https://gitlab.cern.ch/SCNU/LGWallRecon-Claude-Fable52`

To cite the toolkit:

Hengne Li, *LGWallRecon: light-guide inner-wall reconstruction from an entrance-face scan*, software, 2026. Repository: `https://github.com/hengne/LGWallRecon-Claude-Fable52`

When quoting results obtained with it, state the assumptions of the model (perfect specular wall, uniform detector, Lambertian source) and that the reconstructed wall is the minimum-unevenness solution of a non-unique inverse problem.